

QCM Answer Sheet (1–55)

Mark your answer with **X** inside the chosen box.

#	A	B	C	D
1	☐	☐	☐	☐
2	☐	☐	☐	☐
3	☐	☐	☐	☐
4	☐	☐	☐	☐
5	☐	☐	☐	☐
6	☐	☐	☐	☐
7	☐	☐	☐	☐
8	☐	☐	☐	☐
9	☐	☐	☐	☐
10	☐	☐	☐	☐
11	☐	☐	☐	☐
12	☐	☐	☐	☐
13	☐	☐	☐	☐
14	☐	☐	☐	☐
15	☐	☐	☐	☐
16	☐	☐	☐	☐
17	☐	☐	☐	☐
18	☐	☐	☐	☐
19	☐	☐	☐	☐
20	☐	☐	☐	☐
21	☐	☐	☐	☐
22	☐	☐	☐	☐
23	☐	☐	☐	☐
24	☐	☐	☐	☐
25	☐	☐	☐	☐

#	A	B	C	D
26	☐	☐	☐	☐
27	☐	☐	☐	☐
28	☐	☐	☐	☐
29	☐	☐	☐	☐
30	☐	☐	☐	☐
31	☐	☐	☐	☐
32	☐	☐	☐	☐
33	☐	☐	☐	☐
34	☐	☐	☐	☐
35	☐	☐	☐	☐
36	☐	☐	☐	☐
37	☐	☐	☐	☐
38	☐	☐	☐	☐
39	☐	☐	☐	☐
40	☐	☐	☐	☐
41	☐	☐	☐	☐
42	☐	☐	☐	☐
43	☐	☐	☐	☐
44	☐	☐	☐	☐
45	☐	☐	☐	☐
46	☐	☐	☐	☐
47	☐	☐	☐	☐
48	☐	☐	☐	☐
49	☐	☐	☐	☐
50	☐	☐	☐	☐

Exercise I – MCQ Quiz (55 Questions)

Q1. In a single-threaded server, what happens when one request is being processed? (**Que se passe-t-il dans un serveur mono-thread lorsqu'une requête est en cours de traitement ?**)

A) Other clients are immediately processed in parallel (Les autres clients sont immédiatement traités en parallèle)	B) New clients must wait until the current request finishes (Les nouveaux clients doivent attendre la fin de la requête en cours)
C) The server automatically creates new threads (Le serveur crée automatiquement de nouveaux threads)	D) The operating system rejects all new clients (Le système d'exploitation rejette tous les nouveaux clients)

Q2. What is the main weakness of this loop? (**Quelle est la principale faiblesse de cette boucle ?**)

```
while (true) {
    Socket client = serverSocket.accept();
    handleRequest(client);
}
```

A) It processes only one request at a time (Elle traite une seule requête à la fois)	B) It cannot accept sockets (Elle ne peut pas accepter des sockets)
C) It requires a database (Elle nécessite une base de données)	D) It always crashes under one client (Elle plante toujours avec un seul client)

Q3. When 50 clients connect to a single-threaded server, the main problem is usually: (**Lorsque 50 clients se connectent à un serveur mono-thread, quel est le principal problème ?**)

A) Lower memory usage (Utilisation mémoire plus faible)	B) Queue growth and increasing latency (Croissance de la file d'attente et augmentation de la latence)
C) No CPU usage (Aucune utilisation du CPU)	D) Faster response time (Temps de réponse plus rapide)

Q4. The expression $\text{throughput} = \text{requests} / \text{second}$ measures: (**Que mesure l'expression débit = requêtes / seconde ?**)

A) How many requests are completed per second (Le nombre de requêtes traitées par seconde)	B) How many threads exist (Le nombre de threads existants)
C) How many files are open (Le nombre de fichiers ouverts)	D) How much RAM is installed (La quantité de mémoire RAM installée)

Q5. For I/O-bound workloads, increasing the number of threads can help because: (**Pour les charges I/O, pourquoi augmenter le nombre de threads peut aider ?**)

A) Threads can make progress while others are waiting for I/O (Les threads peuvent progresser pendant que d'autres attendent les entrées/sorties)	B) CPU instructions become mathematically free (Les instructions CPU deviennent gratuites mathématiquement)
C) Locks disappear automatically (Les verrous disparaissent automatiquement)	D) Network latency becomes zero (La latence réseau devient nulle)

Q6. What is the CPU-bound trap? (**Quel est le piège des tâches liées au CPU ?**)

A) Adding more threads always gives unlimited speedup (Ajouter plus de threads donne toujours un gain illimité)	B) Adding too many threads to CPU-heavy work may not improve performance (Ajouter trop de threads pour des tâches CPU peut ne pas améliorer les performances)
C) CPU-bound programs never use the CPU (Les programmes CPU n'utilisent jamais le CPU)	D) Thread pools cannot run Java code (Les pools de threads ne peuvent pas exécuter du code Java)

Q7. The engineering formula for thread pool tuning depends strongly on: (**De quoi dépend fortement le réglage d'un pool de threads ?**)

A) The balance between waiting time and compute time (L'équilibre entre temps d'attente et temps de calcul)	B) The color of the terminal (La couleur du terminal)
C) The file name of the Java class (Le nom du fichier Java)	D) The number of comments in the code (Le nombre de commentaires dans le code)

Q8. Amdahl's Law explains: (**Que décrit la loi d'Amdahl ?**)

A) The maximum possible speedup when only part of a program is parallelizable (Le gain maximal possible lorsqu'une partie seulement est parallélisable)	B) How to create a socket (Comment créer un socket)
C) How to write SQL queries (Comment écrire des requêtes SQL)	D) How to disable threads (Comment désactiver les threads)

Q9. If a program has a large sequential part, then even infinite cores will: (**Si une grande partie est séquentielle, que se passe-t-il même avec des cœurs infinis ?**)

A) Give unlimited speedup (Donner un gain illimité)	B) Still be limited by the sequential part (Rester limité par la partie séquentielle)
C) Remove all latency (Supprimer toute latence)	D) Make synchronization unnecessary (Rendre la synchronisation inutile)

Q10. In Little's Law, L represents: (**Dans la loi de Little, que représente L ?**)

A) Average number of items in the system (Nombre moyen d'éléments dans le système)	B) Number of CPU cores (Nombre de cœurs CPU)
C) Length of the source code (Longueur du code source)	D) Number of Java classes (Nombre de classes Java)

Q11. Context switching cost refers to: (**À quoi correspond le coût de changement de contexte ?**)

A) The overhead of switching the CPU from one thread to another (Le coût du passage du CPU d'un thread à un autre)	B) The cost of buying a new computer (Le coût d'achat d'un ordinateur)
C) The time needed to compile comments (Le temps pour compiler les commentaires)	D) The size of the database (La taille de la base de données)

Q12. What does p50 latency represent? (**Que représente la latence p50 ?**)

A) The median response time (Le temps de réponse médian)	B) The maximum possible response time (Le temps de réponse maximal)
C) The number of threads (Le nombre de threads)	D) The number of failed requests only (Le nombre de requêtes échouées uniquement)

Q13.

What does p95 latency help reveal? (**Que révèle la latence p95 ?**)

A) Slow responses experienced by the worst 5 percent of requests (Les réponses lentes subies par les 5% pires requêtes)	B) Only the fastest request (Seulement la requête la plus rapide)
C) The number of CPU cores (Le nombre de cœurs CPU)	D) The Java version (La version de Java)

Q14. A critical section is: (**Qu'est-ce qu'une section critique ?**)

A) Code that accesses shared data and must be protected (Code qui accède à des données partagées et doit être protégé)	B) Code that is never executed (Code qui n'est jamais exécuté)
C) A section used only for printing titles (Une section utilisée uniquement pour afficher des titres)	D) A Java package name (Un nom de package Java)

Q15. In Java, how can you create a thread? (**Comment peut-on créer un thread en Java ?**)

A) By extending Thread or implementing Runnable (En étendant Thread ou en implémentant Runnable)	B) By creating a Socket (En créant un socket)
C) By calling System.exit() (En appelant System.exit())	D) By using SQL queries (En utilisant des requêtes SQL)

Q16. What happens when start() is called on a thread? (**Que se passe-t-il lorsqu'on appelle start() sur un thread ?**)

A) A new thread begins execution and calls run() (Un nouveau thread commence l'exécution et appelle run())	B) The program terminates (Le programme se termine)
C) The thread is deleted (Le thread est supprimé)	D) The CPU is stopped (Le CPU s'arrête)

Q17. Calling run() directly instead of start(): (**Appeler run() directement au lieu de start() :**)

A) Executes in the current thread (no concurrency) (S'exécute dans le thread courant (pas de concurrence))	B) Creates multiple threads (Crée plusieurs threads)
C) Causes a deadlock (Provoque un interblocage)	D) Improves parallelism (Améliore le parallélisme)

Q18. A race condition occurs when: (**Quand une condition de course se produit-elle ?**)

A) Multiple threads access shared data without proper synchronization (Plusieurs threads accèdent à des données partagées sans synchronisation adéquate)	B) Only one thread executes (Un seul thread s'exécute)
C) A program runs too fast (Un programme s'exécute trop rapidement)	D) A variable is constant (Une variable est constante)

Q19. Consider:

```
counter++;
```

Why is this unsafe in multithreading? (**Pourquoi cela est-il dangereux en multithreading ?**)

A) It is not atomic and can be interrupted (Ce n'est pas une opération atomique et elle peut être interrompue)	B) It is too slow (C'est trop lent)
C) It uses too much memory (Cela utilise trop de mémoire)	D) It cannot compile (Cela ne compile pas)

Q20. Race conditions are dangerous because: (**Pourquoi les conditions de course sont-elles dangereuses ?**)

A) They produce unpredictable results (Elles produisent des résultats imprévisibles)	B) They always crash immediately (Elles provoquent toujours un crash immédiat)
C) They prevent compilation (Elles empêchent la compilation)	D) They remove threads (Elles suppriment les threads)

Q21. Mutual exclusion ensures that: (**Que garantit l'exclusion mutuelle ?**)

A) Only one thread accesses a critical section at a time (Un seul thread accède à une section critique à la fois)	B) All threads run simultaneously (Tous les threads s'exécutent simultanément)
C) No thread can run (Aucun thread ne peut s'exécuter)	D) Threads do not exist (Les threads n'existent pas)

Q22. In Java, which keyword provides mutual exclusion? (**Quel mot-clé en Java fournit l'exclusion mutuelle ?**)

A) synchronized (synchronized)	B) public (public)
C) static (static)	D) return (return)

Q23. What is the main purpose of a lock? (**Quel est le but principal d'un verrou ?**)

A) To prevent simultaneous access to shared resources (Empêcher l'accès simultané aux ressources partagées)	B) To speed up printing (Accélérer l'affichage)
C) To reduce memory usage (Réduire l'utilisation mémoire)	D) To stop threads permanently (Arrêter définitivement les threads)
Q24. A deadlock occurs when: (Quand un interblocage se produit-il ?)	
A) Threads wait forever for resources held by each other (Les threads attendent indéfiniment des ressources détenues par d'autres)	B) Threads execute too fast (Les threads s'exécutent trop rapidement)
C) Memory is full (La mémoire est pleine)	D) A program ends (Un programme se termine)
Q25. Which condition is required for deadlock? (Quelle condition est nécessaire pour un interblocage ?)	
A) Circular waiting (Attente circulaire)	B) No threads (Aucun thread)
C) Single variable (Une seule variable)	D) Fast CPU (CPU rapide)
Q26. Example of deadlock: (Exemple d'interblocage :)	
A) Thread A holds Lock1 and waits for Lock2, while Thread B holds Lock2 and waits for Lock1 (Le thread A détient Lock1 et attend Lock2, tandis que le thread B détient Lock2 et attend Lock1)	B) A thread prints a message (Un thread affiche un message)
C) A loop increments a variable (Une boucle incrémente une variable)	D) A file is read (Un fichier est lu)
Q27. One way to avoid deadlock is: (Une façon d'éviter un interblocage est :)	
A) Always acquire locks in the same order (Toujours acquérir les verrous dans le même ordre)	B) Remove all threads (Supprimer tous les threads)
C) Increase memory (Augmenter la mémoire)	D) Use more print statements (Utiliser plus d'instructions d'affichage)
Q28. Deadlock results in: (Un interblocage entraîne :)	
A) System freeze or blocked execution (Blocage du système ou exécution figée)	B) Faster execution (Exécution plus rapide)
C) Lower CPU usage only (Seulement une baisse de l'utilisation CPU)	D) Compilation failure (Échec de compilation)
Q29. What is visibility in concurrency? (Qu'est-ce que la visibilité en concurrence ?)	
A) Whether one thread sees updates made by another thread (Si un thread voit les mises à jour faites par un autre thread)	B) How fast a thread runs (La vitesse d'exécution d'un thread)
C) How many threads exist (Le nombre de threads existants)	D) The size of memory (La taille de la mémoire)
Q30. Without proper synchronization: (Sans synchronisation adéquate :)	
A) Threads may see stale values (Les threads peuvent voir des valeurs obsolètes)	B) Threads always see latest values (Les threads voient toujours les dernières valeurs)
C) Threads stop working (Les threads cessent de fonctionner)	D) Program compiles faster (Le programme compile plus rapidement)
Q31. Which keyword helps ensure visibility in Java? (Quel mot-clé assure la visibilité en Java ?)	
A) volatile (volatile)	B) int (int)
C) class (class)	D) main (main)
Q32. The Java Memory Model defines: (Que définit le modèle mémoire Java ?)	

A) Rules for how threads interact through memory (Les règles d'interaction des threads via la mémoire)	B) Database schemas (Les schémas de bases de données)
C) File system structure (La structure du système de fichiers)	D) Network protocols (Les protocoles réseau)

Q33. Happens-before relationship ensures: (**La relation happens-before garantit :**)

A) Correct ordering and visibility of operations between threads (Un ordre correct et la visibilité des opérations entre threads)	B) Faster compilation (Une compilation plus rapide)
C) Lower memory usage (Une utilisation mémoire réduite)	D) Thread deletion (La suppression des threads)

Q34. The Producer-Consumer problem involves: (**Le problème producteur-consommateur implique :**)

A) Producers generating data and consumers using it (Des producteurs générant des données et des consommateurs les utilisant)	B) Only producers writing files (Seulement des producteurs écrivant des fichiers)
C) Only consumers reading memory (Seulement des consommateurs lisant la mémoire)	D) Threads that do not interact (Des threads qui n'interagissent pas)

Q35. What structure is typically used between producer and consumer? (**Quelle structure est utilisée entre producteur et consommateur ?**)

A) Shared buffer (queue) (Tampon partagé (file d'attente))	B) File system (Système de fichiers)
C) Stack trace (Trace de pile)	D) Compiler (Compilateur)

Q36. If the buffer is full, the producer should: (**Si le tampon est plein, le producteur doit :**)

A) Wait (Attendre)	B) Overwrite data (Écraser les données)
C) Terminate immediately (Se terminer immédiatement)	D) Ignore the buffer (Ignorer le tampon)

Q37. If the buffer is empty, the consumer should: (**Si le tampon est vide, le consommateur doit :**)

A) Wait (Attendre)	B) Create data (Créer des données)
C) Delete the buffer (Supprimer le tampon)	D) Restart the program (Redémarrer le programme)

Q38.

The main challenge in this problem is: (**Quel est le principal défi dans ce problème ?**)

A) Synchronization between threads (La synchronisation entre les threads)	B) Memory allocation only (L'allocation mémoire uniquement)
C) Compilation errors (Les erreurs de compilation)	D) File reading speed (La vitesse de lecture des fichiers)

Q39. Condition variables are typically used with: (**Les variables de condition sont généralement utilisées avec :**)

A) Locks / mutexes (Des verrous / mutex)	B) Arrays (Des tableaux)
C) Files (Des fichiers)	D) Compilers (Des compilateurs)

Q40. What problem do condition variables help solve? (**Quel problème les variables de condition aident-elles à résoudre ?**)

A) Busy waiting inefficiency (L'inefficacité de l'attente active (busy waiting))	B) Syntax errors (Les erreurs de syntaxe)
C) Memory leaks (Les fuites mémoire)	D) Compilation time (Le temps de compilation)

Q41. One necessary condition for deadlock is: (**Quelle est une condition nécessaire pour un interblocage ?**)

A) Mutual exclusion (Exclusion mutuelle)	B) No threads (Aucun thread)
C) Infinite memory (Mémoire infinie)	D) No locks (Aucun verrou)

Q42. Which is a deadlock prevention strategy? (Quelle est une stratégie de prévention des interblocages ?)

A) Lock ordering (Ordonnancement des verrous)	B) Increasing CPU speed (Augmenter la vitesse du CPU)
C) Reducing code size (Réduire la taille du code)	D) Removing variables (Supprimer des variables)

Q43. Breaking circular wait prevents: (Rompre l'attente circulaire permet d'éviter :)

A) Deadlock (Les interblocages)	B) Race conditions (Les conditions de course)
C) Compilation errors (Les erreurs de compilation)	D) Thread creation (La création de threads)

Q44. Resource allocation graphs are used to: (Les graphes d'allocation de ressources sont utilisés pour :)

A) Analyze deadlocks (Analyser les interblocages)	B) Compile programs (Compiler des programmes)
C) Store variables (Stocker des variables)	D) Manage files (Gérer des fichiers)

Q45.

Deadlock avoidance differs from prevention because: (En quoi l'évitement de l'interblocage diffère-t-il de la prévention ?)

A) It dynamically checks safe states (Il vérifie dynamiquement les états sûrs)	B) It removes all locks (Il supprime tous les verrous)
C) It disables threads (Il désactive les threads)	D) It ignores resources (Il ignore les ressources)

Q46. Starvation occurs when: (Quand la famine (starvation) se produit-elle ?)

A) A thread never gets access to resources (Un thread n'obtient jamais l'accès aux ressources)	B) All threads finish quickly (Tous les threads se terminent rapidement)
C) Memory is full (La mémoire est pleine)	D) CPU is idle (Le CPU est inactif)

Q47. Fairness in scheduling means: (Que signifie l'équité dans l'ordonnancement ?)

A) All threads eventually get a chance to execute (Tous les threads finissent par avoir une chance de s'exécuter)	B) Only one thread runs (Un seul thread s'exécute)
C) Threads are removed (Les threads sont supprimés)	D) Execution is random (L'exécution est aléatoire)

Q48. Which mechanism helps reduce starvation? (Quel mécanisme aide à réduire la famine ?)

A) Fair locks / scheduling policies (Verrous équitables / politiques d'ordonnancement équitables)	B) Infinite loops (Boucles infinies)
C) Removing threads (Suppression des threads)	D) Increasing file size (Augmenter la taille des fichiers)

Q49. Priority scheduling may cause: (L'ordonnancement par priorité peut provoquer :)

A) Starvation of low-priority threads (La famine des threads de faible priorité)	B) Compilation errors (Des erreurs de compilation)
C) Memory leaks (Des fuites mémoire)	D) No execution (Aucune exécution)

Q50. Aging technique is used to: (La technique de vieillissement est utilisée pour :)

A) Increase priority of waiting threads over time (Augmenter la priorité des threads en attente avec le temps)	B) Delete old threads (Supprimer les anciens threads)
C) Reduce memory (Réduire la mémoire)	D) Stop execution (Arrêter l'exécution)

Exercise II – Code Tracing and Completion

Code Tracing: Race Condition

Consider the following Java code:

```
class Counter {
    private int count = 0;
```

```
public void increment() {
    count++;
}

public int getCount() {
    return count;
}

public class TestRace {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                c.increment();
            }
        });

        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) {
                c.increment();
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(c.getCount());
    }
}
```

Question II : What is the expected behavior of this program? Should the output always be 2000? Explain briefly. (Quel est le comportement attendu de ce programme ? La sortie doit-elle toujours être 2000 ? Expliquez brièvement.)

.....

.....

.....

.....

.....

.....

.....

.....

Question III: Question III: Code Completion: Thread-Safe Bank Account (Complétion de code : Compte bancaire thread-safe)

We want the following behavior: (On souhaite le comportement suivant :)

- Two threads perform deposits on the same bank account. (Deux threads effectuent des dépôts sur le même compte bancaire.)
- The final balance must always be correct. (Le solde final doit toujours être correct.)
- No race condition should occur. (Aucune condition de course ne doit se produire.)

Complete the missing part of the code. (Complétez la partie manquante du code.)

```
class BankAccount {
    private int balance = 0;

    public void deposit(int amount) {
        // TODO: add the missing synchronization code here

        balance += amount;
    }

    public int getBalance() {
        return balance;
    }
}
```

Required: Rewrite only the `deposit()` method so that the account becomes thread-safe. (Réécrivez uniquement la méthode `deposit()` pour rendre le compte thread-safe.)